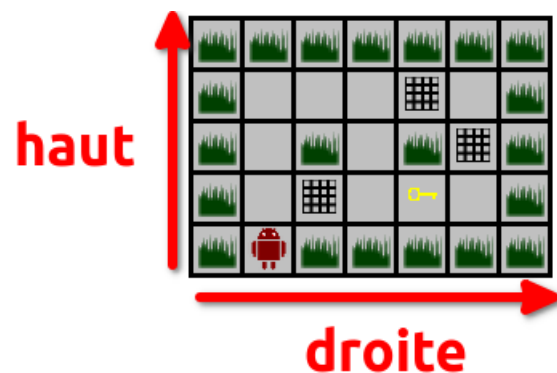


Exercice 1 Manipuler un objet

On dispose d'une classe `Robot` permettant de créer un objet qu'on peut déplacer dans une grille à l'aide des méthodes définies dans son interface :

Méthode	Signature	Spécification
<code>haut</code>	<code>haut(self, n)</code>	Déplace le robot de <code>n</code> cases verticalement vers le haut de la grille
<code>bas</code>	<code>bas(self, n)</code>	Déplace le robot de <code>n</code> cases verticalement vers le bas de la grille
<code>gauche</code>	<code>gauche(self, n)</code>	Déplace le robot de <code>n</code> cases horizontalement vers la gauche de la grille
<code>droite</code>	<code>droite(self, n)</code>	Déplace le robot de <code>n</code> cases horizontalement vers la droite de la grille

Le robot est représenté ainsi . Il ne peut se déplacer que sur les cases dont l'intérieur grisé ne comporte ni fourré, ni piège.



Complétez le programme suivant pour que la robot se déplace jusqu'à la case contenant la clef sans traverser de case avec un fourré ou un piège.

```
nono = Robot() # on crée une instance de la classe Robot
# à vous de déplacer le robot nono jusqu'à la case contenant la clef
...
...
...
...
...
...
...
```

Exercice 2 Vocabulaire et concepts de la POO

On donne ci-dessous la définition d'une classe `Evenement` pour créer des objets représentants des événements caractérisés par l'année, le jour, le mois et une description. On donne ci-dessous une partie de l'implémentation :

```
class Evenement:
    """Classe pour instancier un objet représentant un événement"""

    def __init__(self, a, m, j, d):
        """Construit un objet date en initialisant trois attributs:
        an(type int), mois(type int), jour(type int) et description(
            type str)
        """
        self.an = a # type int
        self.mois = m # type int
        self.jour = j # type int
        self.description = d # type str

    def affiche_date(self):
        """
        Renvoie la date de l'événement sous forme de chaîne(type str)
        au format 'an/mois/jour'
        """
        return str(self.an) + "/" + str(self.mois) + "/" + str(self.
            jour)

    def decale_date(self, da, dm, dj):
        """
        Décale la date de l'événement de da ans, dm mois et dj jours
        """
        # à compléter
        ...
        ...
        ...
        ...
        ...
```

1. Quel terme propre au paradigme de Programmation Orienté Objet, permet de désigner les noms `an`, `mois`, `jour` et `description` ?

.....

2. Quel est le type du paramètre `d` de la méthode `__init__` ?

.....

3. Quelle instruction permet de créer un objet représentant l'événement de description "Concert Schubert" qui aura lieu le 14 Septembre 2023 ?

.....

4. Quel est le type de la valeur renvoyée par la méthode `affiche_date` ?

.....

5. On a créé un objet `match` de la classe `Evenement`. Quelle expression permet d'obtenir la description de l'événement `match` ?

.....

6. Compléter le code de la méthode `decale_date` qui peut modifier la date d'un événement en décalant l'année, le jour et le mois (éventuellement de 0). L'année, le jour et le mois décalés doivent être toujours de type `int` mais on ne vérifiera pas que le décalage produit une date qui existe réellement. On donne un exemple d'utilisation en console interactive :

```
>>> mariage = Evenement(2024, 7, 19, "mariage Sébastien")
>>> mariage.affiche_date()
'2024/7/19'
>>> mariage.decale_date(1, 2, 3)
>>> mariage.affiche_date()
'2025/9/22'
```

Exercice 3 Compléter une classe à partir de son interface

On considère un environnement de jeu vidéo où plusieurs personnages évoluent dans un espace à deux dimensions muni d'un repère. Chaque personnage est caractérisé par :

- ses coordonnées x et y dans le repère
- son nombre de points de vie
- son profil : "elfe", "nain" ...
- son nom

Lorsqu'on crée un personnage, il est initialisé avec des coordonnées, un profil, un nom et un total de points de vie qui dépend de son profil : 100 pour un "elfe" et 50 pour un autre profil.

Des mannes (ou nourritures) sont disséminées dans l'espace de jeu. Une manne est caractérisée par :

- ses coordonnées x et y dans le repère
- sa valeur en nombre de points de vie

Lorsqu'on crée une manne, elle est initialisée avec des coordonnées et sa valeur en points de vie. Une manne peut être consommée par un personnage si et seulement si elle se trouve aux mêmes coordonnées que le personnage. Dans ce cas le personnage ajoute la valeur de la manne à ses points de vie et la valeur de la manne est réduite à zéro.

Un personnage peut attaquer un autre personnage qui se trouve à une distance euclidienne inférieure ou égale à 2 et lui infliger une perte de points de vie qui dépend du profil de l'attaquant : -20 pour un elfe et -10 pour un autre profil. Un personnage meurt lorsque son nombre de points de vie est inférieur ou égal à zéro, après il ne peut plus rien faire.

En début de partie, les mannes et les personnages sont disséminés aléatoirement dans le repère puis les mannes ne bougent plus et les personnages peuvent se déplacer par translation. Le jeu se termine lorsqu'il ne reste plus qu'un seul personnage, désigné vainqueur, les autres étant tous morts.

Partie 1 : une fonction distance

1. Compléter le code de la fonction `distance` qui prend en paramètre les coordonnées $(x1, y1)$ et $(x2, y2)$ de deux points et renvoie leur distance euclidienne calculée avec la formule $\sqrt{(x1 - x2)^2 + (y1 - y2)^2}$.

Vous utiliserez la fonction `sqrt` du module `math`.

```
import math

def distance(x1, y1, x2, y2):
    """Renvoie la distance euclidienne entre
    les points de coordonnées (x1, y1) et (x2, y2)
    """
    ...
```

Partie 2 : une classe Manne

Complétez le code des méthodes `valeur` et `coord` de la classe `Manne` qui permet de créer des objets représentants des mannes dans le jeu.

```
class Manne:
    """
    Classe pour instancier des mannes de points de vie
    """

    def __init__(self, x, y, p):
        """
        Crée une instance de Manne
        au point de coordonnées (x, y)
        avec p points de vie
        """
        self.x = x # abscisse de la manne
        self.y = y # ordonnée de la manne
        self.points = p # points de la manne

    def donne(self):
        """
        Si elle est consommée par un personnage
```

```
l'objet manne lui donne tous ses points
de vie et son total de points est ramené à zéro
"""

p = self.points
self.points = 0
return p

def valeur(self):
    """
    Renvoie le nombre de points de vie de la manne
    """
    ...

def coord(self):
    """
    Renvoie les coordonnées de la manne sous forme de tuple
    """
    ...
```

Partie 3 : une classe Personnage

On donne ci-dessous le début de l'implémentation d'une classe `Personnage` pour le jeu.

```
class Personnage:
    """
    Classe pour instancier des personnages
    """

    def __init__(self, x, y, profil, nom):
        """
        Crée une instance de la classe Personnage
        au point de coordonnées(x, y)
        de profil et de nom passés en paramètres
        """
        self.x = x
        self.y = y
        self.profil = profil
        self.nom = nom
        # à compléter
        ...
        ...
        ...
        ...
```

1. Compléter le code de la méthode `__init__` pour initialiser correctement l'attribut `self.points` : 100 si le profil est égal à "elfe" et 50 sinon.

2. Compléter le code de la méthode de la classe **Personnage** ci-dessous :

```
def est_vivant(self):  
    """  
    Renvoie un booléen indiquant si le personnage est vivant (  
        mort si nombre de points de vie <= 0)  
    """  
    ...
```

3. Compléter le code de la méthode de la classe **Personnage** ci-dessous. On donne un exemple d'interactions entre objets en ANNEXE.

```
def perd_points(self, p):  
    """  
    Le personnage perd p points de vie sur son attribut points  
    cette méthode ne renvoie rien  
    """  
    ...
```

4. On admet que la classe **Personnage** possède aussi :

- une méthode **coord** qui renvoie les coordonnées de l'objet personnage dans le repère du plan sous forme de **tuple** ;
- une méthode **deplace** qui modifie les coordonnées de l'objet personnage dans le repère en leur appliquant une translation.

- (a) Compléter le code de la méthode de la classe **Personnage** ci-dessous. On donne un exemple d'interactions entre objets en ANNEXE.

```
def gagne_points(self, m):  
    """  
    Le personnage gagne les points de vie  
    fournis par la manne m, s'il est vivant  
    et si la manne est aux mêmes coordonnées  
    """  
    assert isinstance(m, Manne)  
    if .....:  
        self.points = self.points + m.donne()  
    else:  
        print("Personnage mort ou manne inatteignable")
```

(b) Compléter le code de la méthode de la classe `Personnage` ci-dessous :

```
def attaque(self, autre):  
    """Attaque sur le personnage autre de classe Personnage.  
    Attaque possible :  
        si autre est vivant  
        et s'il est à une distance euclidienne <= 2  
    Si le personnage a le profil 'elfe' son attaque fait  
        perdre 20 points à l'autre  
    sinon elle fait perdre 10 points à l'autre  
    Il ne se passe rien si l'attaque n'est pas possible"""  
    assert self.est_vivant()  
    ...  
    ...  
    ...  
    ...  
    ...  
    ...  
    ...  
    ...  
    ...
```

ANNEXE

Voici un exemple d'interactions entre objets :

```
orange = Manne(-6, 8, 30)  
legolas = Personnage(5, 4, "elfe", "legolas")  
legolas.deplace(-11, 4)  
assert legolas.coord() == orange.coord()  
legolas.gagne_points(orange)  
assert legolas.points == 130  
assert orange.valeur() == 0  
gimli = Personnage(-7, 8, "nain", "gimli")  
assert gimli.points == 50  
legolas.attaque(gimli)  
assert gimli.points == 30
```